



FOR JAVA DEVELOPERS MOVING TO NESTJS

10,000 sessions. No expiry.

Spring Boot to NestJS, episode 12: CORS, cookies and sessions

The concept transfers. The infrastructure defaults do not.

A session is a cookie carrying an identifier, plus state on the server, on both stacks.

Spring's servlet container provides the server side with a 30 minute timeout. In NestJS on Express you install express-session, and inherit its MemoryStore and its lack of expiry.

CORS is enforced by the browser on both, in two paths, and which origins you trust is your decision on both.

In scope

- express-session in NestJS on Express, and its defaults
- The Set-Cookie headers both stacks send
- CORS for a simple request and for a preflighted one
- Credentialed CORS: reflected origins, Spring's wildcard rule

Not in scope

- CSRF protection: a separate subject
- What browsers do with a missing SameSite

Preflight pass or fail is the Fetch standard's check applied by the script, not a real browser.

01 The measurements

NODE_ENV=development	no warning
NODE_ENV=production	MemoryStore warning
10,000 ONE-TIME VISITORS	10,000 sessions held, 2 s later 10,000
cookie maxAge / expires	null / null
SPRING SESSION TIMEOUT	1800 s, reported
SIMPLE GET, ANY CONFIG	handler ran 1x
PREFLIGHTED PUT, none / listed origin	real PUT not sent
origin: true + credentials	reflects https://evil.example

Nothing configured unless stated.

02 Register a session, and choose its store and expiry

express-session keeps sessions in a MemoryStore unless you give it another store, and sets no cookie maxAge unless you do. Choose both before production.

 sessions.ts (repository)

```
import session from 'express-session';

app.use(session({ secret: 'demo-only', resave: false, saveUninitialized: false }));
```

 the Spring side, for comparison

```
@GetMapping("/visit")
public Map<String, Object> visit(HttpSession session) { ... }

# default: server.servlet.session.timeout=30m
```

MemoryStore "is purposely not designed for a production environment. It will leak memory under most conditions, does not scale past a single process"

[express-session 1.19.0 README](#), read 2026-09-30.

cookie.maxAge: "By default, no maximum age is set."

[express-session 1.19.0 README](#), read 2026-09-30.

server.servlet.session.timeout default 30m

[spring-boot-web-server 4.1.1 configuration metadata](#), read 2026-09-30.

03 CORS: two browser paths

A simple GET is sent for real; CORS decides whether the page may read the answer. A preflighted request is checked with OPTIONS first, and if that fails the real request is never sent. CORS is browser enforcement, not server side authorization.

```
origins you trust, by name
```

```
app.enableCors({ origin: ['https://app.example'], credentials: true });
```

Simple GET from https://evil.example: handler ran once under all four configurations; preflighted PUT: not sent with nothing configured or with a listed origin

Run on a real machine, 2026-09-30, by scripts/verify-sessions.sh in the course repository.

origin: true with credentials: true reflected https://evil.example with credentials allowed; Spring allowedOriginPatterns("*") did the same

Run on a real machine, 2026-09-30, by scripts/verify-sessions.sh in the course repository.

Before you ship sessions in NestJS

1. Replace MemoryStore with a real store
2. Set cookie.maxAge, and secure in production
3. Set NODE_ENV=production so the package can warn you
4. List trusted origins by name when credentials are on
5. Never origin: true with credentials: true
6. Treat CORS as browser policy, not authorization



Everything else

- | | |
|--|--|
|  Site
https://code-with-sam-dev.github.io |  Facebook
https://www.facebook.com/codewithsamdev |
|  YouTube
https://www.youtube.com/@CodewithSam-Dev |  GitHub
https://github.com/code-with-sam-dev |
|  LinkedIn
https://www.linkedin.com/company/code-with-sam-dev |  Buy me a coffee
https://buymeacoffee.com/codewithsam |
|  x
https://x.com/CodeWithSamDev |  Buy me sadza
https://buymesadza.com/codewithsam |
|  TikTok
https://www.tiktok.com/@codewithsamdev | Every measurement, one command: scripts/verify-sessions.sh
https://github.com/code-with-sam-dev/spring-to-node-express-session |
|  Instagram
https://www.instagram.com/codewithsamdev | |

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-09-30 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.